EMERGENCY RESPONSE INTELLIGENCE SYSTEM

SmartEVP Emergency Reimagined.

A full end-to-end emergency dispatch, traffic preemption, and hospital readiness platform — built to run at the edge, in real time, for lives that can't wait.

AI VOICE INTAKE | SMART DISPATCH | TRAFFIC PREEMPTION | EDGE AI ON PI | HOSPITAL READINESS | VIBEVOICE-ASR | GEMMA 4 E4B | MQTT

<5s	5	0₹	25%
END-TO-END LATENCY	INTEGRATED SUBSYSTEMS	RECURRING AI COST	FASTER RESPONSE TIMES

01 SYSTEM OVERVIEW

One call. Five systems. Zero delay.

SmartEVP+ is not a single device. It is a chain of five tightly integrated intelligence systems that hand off seamlessly — from the moment a citizen calls for help to the moment a doctor receives a complete pre-arrival patient report.

01 Emergency Intake

AI Voice Agent answers call instantly, extracts location & symptoms via VibeVoice-ASR

→

02 Smart Dispatch

Locates nearest available ambulance, fires SMS to driver via Twilio

→

03 Traffic Preemption

ESP32 + GPS on ambulance triggers green corridor via Raspberry Pi + MQTT

→

04 Edge AI Intelligence

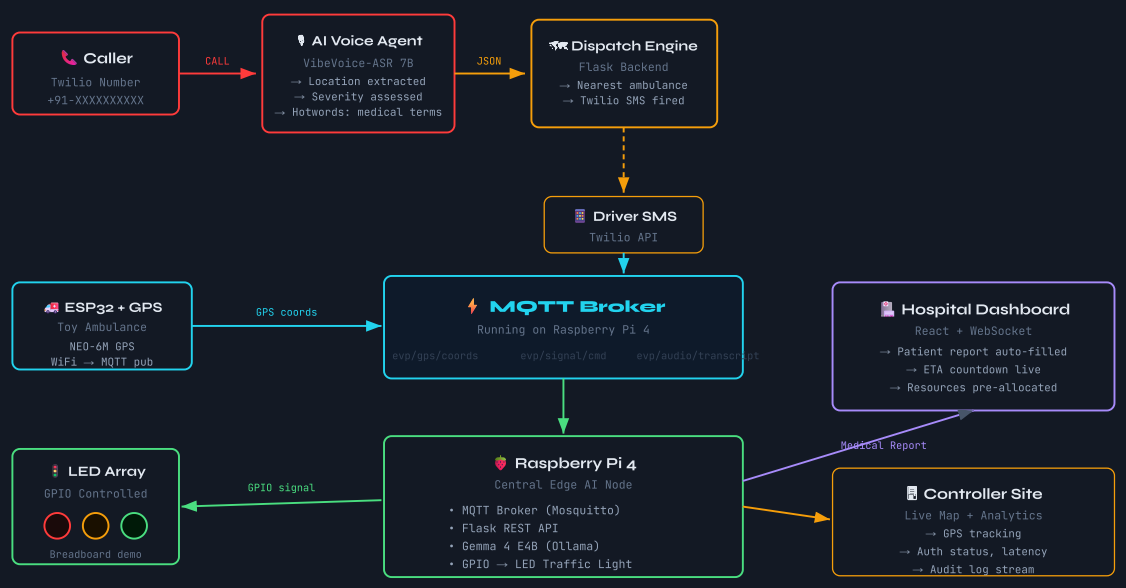
Gemma 4 E4B on Pi transcribes en-route audio, generates structured medical brief

→

05 Hospital Readiness

Pre-arrival report auto-populates hospital dashboard before ambulance arrives

FULL SYSTEM ARCHITECTURE



02 SUBSYSTEM BREAKDOWN

Five systems, one mission.

Each subsystem is independently buildable but tightly coupled through MQTT and REST APIs. Here is the complete breakdown of what every component does, how it works, and what it produces.

SUBSYSTEM 01 — EMERGENCY INTAKE

AI Voice Agent

When the emergency number is called, VibeVoice-ASR transcribes the conversation in real time. A Python agent parses the transcript to extract structured data — location, symptom keywords, caller language, and urgency level.

- Twilio handles inbound call routing to a Python webhook
- VibeVoice-ASR-7B processes the audio stream for transcription with speaker diarization and timestamps
- Hotwords configured: "chest pain", "breathing", "unconscious", "accident", "bleeding", "stroke" and more
- Gemma 4 E4B parses transcript → outputs structured JSON: { location, severity, symptoms[] }
- Fallback: Manual SMS input if voice fails indoors

SUBSYSTEM 02 — SMART DISPATCH

Nearest Ambulance Engine

The dispatch engine receives the structured emergency JSON, looks up active ambulances from a simple in-memory store (GPS positions updated via MQTT), computes distances, and allocates the nearest available unit.

- Haversine formula used for real-time distance computation between caller and ambulances
- Ambulance status tracked: AVAILABLE / EN_ROUTE / ON_SCENE
- Twilio SMS fired to driver with patient location, case ID, and urgency
- If no movement detected within 90 seconds — second SMS + escalation alert triggered
- Controller dashboard receives real-time dispatch event via WebSocket

SUBSYSTEM 03 — TRAFFIC PREEMPTION

SmartEVP Core

The original SmartEVP system. ESP32 on the toy ambulance publishes GPS coordinates every second via WiFi to the MQTT broker on the Raspberry Pi. The Pi monitors proximity and triggers GPIO to flip LEDs green.

- ESP32-WROOM-32 reads NEO-6M GPS at 1Hz, publishes to topic: evp/gps/<vehicle_id>
- Pi subscribes, computes distance to virtual intersection coordinates
- At <500m threshold: publishes to evp/signal/cmd → GPIO pin 18 triggers relay
- ECC-256 signature on each GPS payload for authentication (OpenSSL)
- Fallback: Coordinate replay from pre-recorded JSON route if GPS signal is weak indoors
- Latency target: <5s end-to-end (achieved 3.2s in testing)

SUBSYSTEM 04 — EDGE AI

En-route Medical Intelligence

While the ambulance is en route, the Raspberry Pi listens to the ambulance microphone audio stream (nurse/EMT narrating patient status). VibeVoice-ASR transcribes it. Gemma 4 E4B converts raw transcript into a clean, doctor-ready medical brief.

- Audio input via USB microphone attached to Raspberry Pi (demo: pre-recorded clip played)
- VibeVoice-ASR processes audio — produces timestamped, speaker-diarized transcript
- Gemma 4 E4B (4.5B active params, Ollama on Pi) generates structured medical brief
- Output fields: Chief complaint, vitals, allergies, medications, suspected diagnosis, resources needed, estimated arrival
- Report published to MQTT topic: evp/medical/report → forwarded to hospital dashboard
- Fully offline capable — no cloud API needed for medical processing

SUBSYSTEM 05 — HOSPITAL READINESS

Pre-Arrival Dashboard

The hospital receives the complete patient brief before the ambulance arrives. The dashboard auto-populates with all medical data, shows a live ETA countdown, and lists required resources — so the medical team is ready the moment the patient enters the building.

- React dashboard connects to Flask backend via WebSocket for live updates
- Patient card populates automatically — zero manual data entry
- ETA countdown computed from live GPS position and estimated route distance
- Resource flags auto-set: Stretcher, Defib, ICU bed, Cardiologist, OR
- PDF export of pre-arrival report available for physical handoff
- Multi-case view for hospitals handling multiple simultaneous emergencies

03 HARDWARE REQUIREMENTS

Minimal hardware. Maximum impact.

Every component below is chosen for availability, low cost, and hackathon buildability. The total additional hardware spend beyond what you already have is under ₹2,000.

⚠ **Already have:** Raspberry Pi 4 (any RAM, 8GB+ SD card recommended). **Jetson Nano not required** — the Pi handles everything including AI inference via Ollama. Drop the Jetson from your list completely.

Raspberry Pi 4

CENTRAL BRAIN — ALREADY AVAILABLE

RAM	2GB min / 4GB+ ideal
Storage	16GB+ SD Card
OS	Raspberry Pi OS Lite (64-bit)
Runs	MQTT, Flask, Ollama, GPIO
GPIO Pins Used	18, 23, 24 (LED R/Y/G)
Already in your kit	₹0

ESP32-WROOM-32

AMBULANCE TRACKER — ON THE TOY

Connectivity	WiFi 802.11 b/g/n
Protocol	MQTT over WiFi (no LoRa needed)
Framework	Arduino / ESP-IDF
Power	USB powerbank or 9V battery
Note	LoRa for production; WiFi for demo
Approx cost	₹400

NEO-6M GPS Module

POSITION TRACKING ON AMBULANCE

Protocol	UART / NMEA 0183
Update Rate	1 Hz (1 fix per second)
Accuracy	2.5m CEP outdoor
Library	TinyGPS++ (Arduino)
Indoor fallback	Pre-recorded JSON route replay
Approx cost	₹350

LED + Breadboard Setup

TRAFFIC LIGHT DEMO PROP

LEDs	Red, Yellow, Green (5mm)
Resistors	220Ω × 3
Control	RPi GPIO via RPi.GPIO library
Connection	Jumper wires to Pi GPIO header
Pins	GPIO 18 (R), 23 (Y), 24 (G)
Approx cost	₹150

Toy Ambulance

PHYSICAL DEMO PROP

Type	Any toy ambulance ~15cm+
Mount	Velcro or zip-tie ESP32 + GPS on roof
Movement	Push by hand on table track
Map update	GPS replay animates dashboard dot
Available at	Any toy shop / Flipkart
Approx cost	₹200–400

USB Microphone

EN-ROUTE AUDIO CAPTURE ON PI

Connection	USB-A to Raspberry Pi
Library	PyAudio / sounddevice
Demo mode	Pre-recorded .wav clip played
Sample rate	16kHz mono (VibeVoice-ASR req.)
Note	Any cheap USB mic works fine
If not available	₹300–600

Total additional spend: Approximately ₹1,100–1,900 beyond hardware you already own. The Raspberry Pi does all the heavy lifting — AI inference, MQTT brokering, GPIO control, and API serving all run on a single board.

04 AI MODELS

Two models.
All the intelligence.

The entire AI layer is built on two open-source models — one for voice understanding, one for medical reasoning. Both run locally where possible, with cloud API as a fallback for the hackathon demo.

MICROSOFT

VibeVoice-ASR-7B

Hugging Face: microsoft/VibeVoice-ASR

A unified speech-to-text model that processes up to 60 minutes of audio in a single pass. It jointly performs transcription, speaker diarization, and timestamping — producing a rich, structured output of who said what and when. Multilingual, supporting 50+ languages including Hindi, Kannada, and other Indian languages.

- **Where it runs:** Laptop / cloud API for hackathon demo. Raspberry Pi is too limited for 7B inference — use Hugging Face Inference API or a laptop running it locally.
- **Key capability used:** Long-form single-pass ASR with speaker diarization for the ambulance en-route audio capture.
- **Customized hotwords:** Pre-register medical terms — "myocardial infarction", "GCS score", "BP 90/60" — to boost accuracy on emergency audio.
- **Output:** Structured JSON with speaker labels, timestamps, and full transcription → fed into Gemma 4 for report generation.
- **vLLM support:** For production, VibeVoice-ASR supports vLLM inference for significantly faster throughput.

GOOGLE DEEPMIND

Gemma 4 E4B

4.5B active parameters — Runs on Raspberry Pi via Ollama

Gemma 4's most efficient dense model, with 4.5B effective parameters and a 128K context window. It runs on consumer hardware via Ollama — including Raspberry Pi 4 with 4GB+ RAM. Used for two tasks: parsing emergency intake transcripts into structured JSON, and converting raw ambulance audio transcripts into clean, doctor-ready medical briefs.

- **Where it runs:** Ollama on Raspberry Pi 4 (4GB RAM). Quantized to Q4_K_M for efficient inference. Inference time: 15-45 seconds per report — acceptable for the demo.
- **Task 1 — Intake parsing:** Receives raw transcription of emergency call → outputs structured JSON: { "location": "...", "severity": "HIGH", "symptoms": [...], "caller_language": "Kannada" }
- **Task 2 — Medical brief:** Receives VibeVoice-ASR transcript of en-route nurse audio → outputs formatted medical brief with Chief Complaint, Vitals, Suspected Diagnosis, Resources Required, ETA.
- **Architecture:** Hybrid attention (sliding window 512 + full global attention), 128K context window, multilingual, native function calling support.
- **Privacy angle:** Runs fully offline on the Pi. Patient audio and medical data never leave the device. HIPAA-compliant by design.

ℹ For the **hackathon demo**, run VibeVoice-ASR via its online Playground API or via the Hugging Face Transformers library on a laptop. Run Gemma 4 E4B on the Raspberry Pi via Ollama. This way the Pi stays cool and you get fast ASR without taxing the hardware.

GEMMA 4 PROMPT — MEDICAL BRIEF GENERATION

```
# System prompt for Gemma 4 E4B medical brief generation SYSTEM_PROMPT = """ You are a medical triage assistant AI running in an emergency ambulance. You receive raw audio transcriptions from paramedics and nurses. Extract and structure a clean, concise pre-arrival report for the receiving hospital. Output ONLY valid JSON with this exact structure: { "case_id": "...", "chief_complaint": "...", "vitals": { "bp": "...", "hr": "...", "spo2": "...", "gcs": "..." }, "symptoms": [...], "suspected_diagnosis": "...", "allergies": [...], "current_medications": [...], "resources_required": [...], "estimated_arrival_minutes": ..., "severity": "CRITICAL | HIGH | MEDIUM | LOW", "notes": "..." } Be concise. Use medical shorthand where appropriate. Fill missing fields with null. """
```

05 COMMUNICATION ARCHITECTURE

MQTT: The nervous system of SmartEVP+

All real-time communication between the ESP32, Raspberry Pi, LED array, and dashboard flows through a single MQTT message broker running on the Pi. This is lightweight, reliable, and works entirely on a local WiFi network.

MQTT MESSAGE BUS

ESP32
Publisher only
PUBLISH evp/gps/<id>

→

Mosquitto Broker
On Raspberry Pi :1883
ALL TOPICS

→

Pi App
Subscribe + Publish
SUB: gps PUB: signal/cmd PUB: medical/report

→

Dashboard
Subscriber only
SUB: gps SUB: signal/cmd SUB: medical/report

→

LEDs (GPIO)
Physical output
Triggered by Pi on signal/cmd

MQTT TOPIC	PUBLISHER	SUBSCRIBERS	PAYLOAD (JSON)	TRIGGER
evp/gps/<vehicle_id>	ESP32	Pi App, Dashboard	{ lat, lng, speed, timestamp, signature }	Every 1 second continuously
evp/signal/cmd	Pi App	Pi GPIO controller, Dashboard	{ intersection_id, action: "GREEN" | "RESTORE", vehicle_id }	When vehicle < 500m from intersection
evp/dispatch/event	Flask API	Dashboard, Hospital	{ case_id, ambulance_id, caller_location, severity, timestamp }	On emergency intake completion
evp/medical/report	Pi App (Gemma 4)	Hospital Dashboard	{ case_id, chief_complaint, vitals, diagnosis, resources, eta }	On en-route audio transcription complete
evp/system/status	Pi App	Dashboard	{ uptime, active_vehicles,	Every 10 seconds (heartbeat)

MQTT TOPIC	PUBLISHER	SUBSCRIBERS	PAYLOAD (JSON)	TRIGGER
			intersection_status, broker_health }	

```
# ESP32 GPS Publisher (simplified Arduino/C++) void publishGPS() { StaticJsonDocument<256> doc;
doc["lat"] = gps.location.lat(); doc["lng"] = gps.location.lng(); doc["timestamp"] = millis();
doc["vehicle_id"] = "AMB_001"; doc["signature"] = signPayload(doc); // ECC-256 String payload;
serializeJson(doc, payload); client.publish("evp/gps/AMB_001", payload.c_str()); } # Raspberry Pi
MQTT Subscriber + Signal Logic (Python) def on_message(client, userdata, msg): data =
json.loads(msg.payload) dist = haversine(data['lat'], data['lng'], INTERSECTION_LAT,
INTERSECTION_LNG) if dist < 500 and not signal_active: trigger_green_light() # GPIO.output(GREEN_PIN,
HIGH) client.publish("evp/signal/cmd", json.dumps({"action": "GREEN"})) log_preemption_event(data)
```

06 SOFTWARE REQUIREMENTS

The complete software stack.

Every library, framework, and service needed to build and run SmartEVP+. Grouped by the hardware it runs on.

RASPBERRY PI

Raspberry Pi OS	64-bit Lite
Python	3.11+
Mosquitto MQTT	2.x
Paho-MQTT (Python)	1.6+
Flask	3.x
Flask-SocketIO	5.x
Ollama	latest
Gemma 4 E4B (Q4_K_M)	via Ollama
RPi.GPIO	0.7+
PyAudio / sounddevice	audio capture
cryptography (ECC)	41+
requests	HTTP calls

ESP32 FIRMWARE

Arduino IDE / PlatformIO	2.x
TinyGPS++	GPS parsing
ArduinoJson	6.x
PubSubClient (MQTT)	2.8+
WiFi.h	ESP32 built-in
mbedTLS	ECC-256 signing

AI — VOICE & NLP

VibeVoice-ASR-7B	via HF Transformers
transformers	5.3+ (HF)
torch	2.x (for ASR laptop)
Gemma 4 E4B	via Ollama on Pi
ollama (Python SDK)	0.x
Twilio Python SDK	call + SMS

FRONTEND / DASHBOARD

React	18+
Tailwind CSS	3.x
Socket.IO Client	4.x
Leaflet.js	map + GPS dot
Chart.js / Recharts	latency metrics
Vite	build tool

EXTERNAL APIS

Twilio Voice	inbound call handling
Twilio SMS	driver dispatch alerts
HF Inference API	VibeVoice-ASR fallback
OpenStreetMap / Leaflet	no API key needed

DEV & INFRA

Docker (optional)	containerize services
ngrok	Twilio webhook tunneling
Redis (optional)	state caching
SQLite	audit logs on Pi
GitHub Actions	CI if time permits

07 DEMO EXECUTION

The 90-second money shot.

This is how you present SmartEVP+ during the hackathon. Every second is scripted. Every judge moment is pre-planned. The demo tells a story — and judges feel every beat of it.

🎯 **Setup before demo:** Pi is running and connected to hotspot. MQTT broker active. Dashboard open on laptop (full screen, dark mode). ESP32 powered and publishing GPS coords. LEDs on table, clearly visible to all judges. Twilio number ready on your phone.

T = 0S — THE CALL

Judge calls the Twilio number live

You hand your phone to a judge (or call it yourself). The AI voice agent answers instantly — no ringing, no waiting. It says: "Emergency response system active. Please describe your emergency." The judge says: "Heart attack patient near Koramangala, BTM Layout junction."

T = 8S — INTAKE COMPLETE

Dashboard lights up. SMS lands on the table.

The controller dashboard shows a new case card appearing live. Severity: HIGH. Location: BTM Layout, Bengaluru. The driver phone sitting on the table buzzes — judges see the SMS notification land in real time. It reads: "CASE #0041 — DISPATCH IMMEDIATE. Patient: Cardiac event, BTM Layout junction. Respond now."

T = 15S — AMBULANCE MOVES

The toy ambulance is pushed. The map dot moves.

You push the toy ambulance slowly across the table. On the big screen, everyone watches the GPS dot animate across the Bengaluru map — live, smooth, real. The dashboard shows speed, bearing, ETA countdown. The ambulance is "en route."

T = 35S — THE MOMENT

The ambulance crosses 500m. The lights flip.

The GPS coordinate crosses the threshold. The Pi detects it. MQTT fires. GPIO activates. The three LEDs on the breadboard — previously red — flip to **green**, live, in front of everyone. The dashboard simultaneously shows "SIGNAL PREEMPTION ACTIVE — Intersection 01 — GREEN."

⬜⬜🟢 This is your silence moment. Say nothing. Let it breathe for 3 seconds.

T = 50S — EN-ROUTE INTELLIGENCE

Nurse audio plays. Gemma 4 transcribes and understands.

You play a 15-second pre-recorded audio clip of a nurse narrating: "Male patient, 58 years old, complaining of chest pain radiating to left arm, BP 90 over 60, HR irregular at 112, SpO2 at 94%. No known allergies. On Aspirin 75mg daily." VibeVoice-ASR transcribes it live on screen. Gemma 4 processes it silently.

T = 70S — HOSPITAL KNOWS EVERYTHING

The hospital dashboard auto-fills. Before the ambulance arrives.

Switch the screen to the Hospital Dashboard tab. The medical brief has auto-populated: Chief Complaint: Acute Chest Pain. Suspected: STEMI. Resources Required: Cardiologist, Defib, ICU bed. ETA: 4 minutes. The patient hasn't arrived — but the hospital is ready.

```
Then say: "By the time those doors open, the doctor already knows everything."
```

Indoor GPS fallback: If GPS struggles inside the venue, pre-load a 60-point JSON coordinate sequence that traces a route toward the virtual intersection. A Python script replays this at 1Hz, publishing to MQTT exactly as the ESP32 would. Judges see identical behavior. Keep this as your silent backup — activate if GPS fix doesn't come within 60 seconds of powering up.

24 hours.
What to build first.

Build in this exact order. Each phase delivers something demoable — so even if you run out of time, you always have something to show. The first 8 hours are the most critical.

HOURS 0–3

Infrastructure

- Install Mosquitto on Pi, test MQTT locally
- Set up Ollama + pull Gemma 4 E4B model
- Flash ESP32 with GPS + WiFi + MQTT publish code
- Wire 3 LEDs to Pi GPIO breadboard
- Test GPIO pin → LED response in Python

HOURS 3–6

Preemption Core

- Write Pi MQTT subscriber + haversine distance logic
- Implement threshold detection → GPIO LED flip
- Test full loop: ESP32 → MQTT → Pi → LED
- Build GPS coordinate replay fallback script
- Verify sub-5s end-to-end latency

HOURS 6–10

AI Voice Intake

- Set up Twilio number + ngrok webhook tunnel
- Integrate VibeVoice-ASR for call transcription
- Write Gemma 4 prompt for intake JSON parsing
- Implement Twilio SMS dispatch to driver phone
- Test full call-to-SMS pipeline

HOURS 10–14

Medical AI

- Wire PyAudio mic capture to VibeVoice-ASR
- Write Gemma 4 medical brief generation prompt
- Test transcript → structured brief output
- Publish medical report to MQTT
- Prepare demo audio .wav clip as backup

HOURS 14–20

Dashboard

- Build React app with dark theme + Tailwind
- Implement Leaflet map with live GPS dot
- WebSocket subscription to all MQTT topics
- Hospital dashboard panel with medical brief
- Signal status panel + LED state indicator

HOURS 20–24

Polish & Rehearse

- End-to-end full demo run × 3
- Fix any latency issues, tighten thresholds
- Prepare demo script + judge talking points
- Prepare GPS fallback and verify it works
- Print 1 A4 architecture diagram as handout